TP 1 : Prise en main de l'émulateur 8086 et programmation en assembleur

1.1 Modes d'adressage

Que remarquez-vous concernant le contenu des différents registres ?

Plusieurs registres peuvent contenir la même donnée lorsqu'ils accèdent à la même adresse mémoire, même si le mode d'adressage utilisé est différent.

Que remarquez-vous concernant la mémoire ?

(Espace de réflexion ou observation libre)

INSTRUCTION	CODE MACHINE	ADRESSE LOGIQUE	ADRESSE PHYSIQUE	MODIFICATIONS	COMMENTAIRE / MODE D'ADRESSAGE
`org 100h`	-	-	-	-	Adresse de départ du code/des données
`mov ax, 15h`	B81500	0700:0100	07100	**AX, IP**	Charger ax avec 15h **Immédiat de registre**
`mov bx, 2502h`	BB0225	0700:0103	07103	**BX, IP**	Charger bx avec 2502h **Immédiat de registre**
`mov [0300h], ax`	A30003	0700:0106	07106	**[0300h], IP**	Stocke le contenu de ax à l'adresse mémoire DS:0300h **Immédiat de la mémoire**
`mov [0302h], bx`	891E0203	0700:0109	07109	**[0302h], [0303h], IP**	Stocke le contenu de bx à l'adresse mémoire DS:0302h **Immédiat de la mémoire**
`mov [0304h], 35h`	C606040335	0700:010D	0710D	**[0304h], IP**	Stocke l'octet 35h à l'adresse DS:0304h **Immédiat de la mémoire**
`mov bx, 0300h`	BB0003	0700:0112	07112	**BX, IP**	Charger bx avec 0300h (pointeur) **Immédiat de registre**
`mov cx, [bx]`	8B0F	0700:0115	07115	**CX, IP**	Charger cx avec le contenu de la zone mémoire DS:bx **Basé**
`mov dx, [bx+2]`	8B5702	0700:0117	07117	**DX, IP**	Charge dx avec DS:bx+2 **Basé**
`mov ax, [bx]+4`	8B4704	0700:011A	0711A	**AX, IP**	Charger ax avec DS:0300h + 4 **Basé**
`mov si, bx`	8BF3	0700:011D	0711D	**SI, IP**	Charger le registre SI avec 0300h **Par registre**
`mov di, [si]`	8B3C	0700:011F	0711F	**DI, IP**	Charger DI avec le contenu du mot à DS:0300h **Indexé**
`ret`	C3	0700:0121	07121	**SP, IP**	Fin du programme

1.2.1 Opérations arithmétiques

Visualiser le contenu du registre d'états (flags). Conclure.

Les registres **OF** et **CF** sont égaux à 1 car le résultat de la multiplication dépasse la capacité du registre AX, ce qui nécessite l'utilisation du registre DX pour stocker les bits de poids fort.

Le mode Single Run :

Le mode ***single run*** effectue toutes les instructions automatiquement les unes après les autres, sans s'arrêter après chaque instruction.

Première façon de déclarer les variables

```
; Fixer le registre de segment de données "DS" sur 100h pour
; pouvoir sauvegarder les résultats dans la mémoire
; à partir de l'adresse logique 100h:300h
mov ax, 100h
mov ds, ax

mov ax, var1
add ax, var2
mov [0300h], ax

mov ax, var1
sub ax, var2
mov [0300h]+2, ax

mov ax, var1
mul var2
mov [0300h]+4, ax
mov [0300h]+6, dx

mov ax, var1
mov bx, var2

; On met dx à zéro pour éviter les erreurs car la DIV utilise DX
; comme la "suite" de AX pour former un seul grand nombre de 32 bits
mov dx, 0
div bx
mov [0300h]+8, ax
mov [0300h]+10, dx

ret

; Première façon de déclarer les variables
var1 DW 1525h
var2 DW 1000h
```

Deuxième façon de déclarer les variables

```
; Au lieu d'utiliser mov ax, 100h puis mov ds, ax
; On peut écrire org 0h
org 0h

jmp start

; Deuxième façon de déclarer les variables
var1 DW 1525h
var2 DW 1000h

start:
mov ax, var1
add ax, var2
mov [0300h], ax

mov ax, var1
sub ax, var2
mov [0300h]+2, ax

mov ax, var1
mul var2
mov [0300h]+4, ax
mov [0300h]+6, dx

mov ax, var1
mov bx, var2
mov dx, 0
div bx
mov [0300h]+8, ax
mov [0300h]+10, dx

ret
```

Troisième façon : Utilisation des segments (Data & Code)

```
; Il ne faut pas utiliser "org" lorsqu'on utilise les segments
; Troisième façon de déclarer les variables :
; En utilisant le data segment, on commence à partir de l'adresse 0710h:100h

data segment
    var1 DW 1525h
    var2 DW 1000h
data ends

code segment
    assume ds:data, cs:code

start:
    mov ax, data
    mov ds, ax

    mov ax, var1
    add ax, var2
    mov [0300h], ax

    mov ax, var1
    sub ax, var2
    mov [0300h]+2, ax

    mov ax, var1
    mul var2
    mov [0300h]+4, ax
    mov [0300h]+6, dx

    mov ax, var1
    mov bx, var2
    mov dx, 0
    div bx
    mov [0300h]+8, ax
    mov [0300h]+10, dx

    hlt

code ends
end start
```

1.2.2 Opérations logiques

Visualiser le contenu du registre d'états (flags). Conclure.

- Les drapeaux **ZF** et **PF** sont égaux à 1 à cause de l'opération **AND** .
- Les drapeaux **SF** et **PF** sont égaux à 1 à cause de l'opération **OR** .
- Les drapeaux **SF** et **PF** sont égaux à 1 à cause de l'opération **XOR** .
- Les drapeaux **AF** et **CF** sont égaux à 1 à cause de l'opération **NEG** .

Le mode single run effectue toutes les instructions automatiquement les unes après les autres, sans s'arrêter après chaque instruction.

```
org 100h

mov al, var1
and al, var2
mov result1, al

mov al, var1
or al, var2
mov result2, al

mov al, var1
xor al, var2
mov result3, al

mov al, var1
not al
mov result4, al

mov al, var1
neg al
mov result5, al

ret

var1 db 11001100b
var2 db 00110011b

; Données initiales aléatoires
result1 db 10101010b
result2 db 10101010b
result3 db 10101010b
result4 db 10101010b
result5 db 10101010b
```

1.3 Boucles et instructions de branchement (Tableaux)

Quelles sont les adresses logiques et physiques des variables min et max ?

Min : Logique 0710:0009 | Physique 07109

Max : Logique 0710:0008 | Physique 07108

Visualiser l'évolution du contenu. Conclure.

À chaque itération sur la table, les variables **max** et **min** sont mises à jour si nécessaire. À la fin de la boucle, max contient la plus grande valeur de la table et min la plus petite.

Recherche sur tableau 8 bits (Octets)

```
data segment
    table db 15, 20, 10, 11, 2, 35, 60, 30
    max db ?
    min db ?
    taille db 8
data ends

code segment
    assume ds:data, cs:code

start:
    mov ax, data
    mov ds, ax

    ; On met l'adresse de la variable table dans SI
    lea si, table

    ; On met la donnée dont l'offset est contenu dans SI
    mov al, [si]

    ; Les variables max, min et taille sont des octets (8 bits)
    ; Donc on utilise 8 bits du registre ax (al) et de cx (cl)
    mov max, al
    mov min, al
    mov cl, taille
    dec cl

    ; SI s'incrémente pour accéder au 2ème élément
    inc si

boucle:
    mov al, [si]
    cmp al, max

    ; Si le nombre actuel est ⩽ au max, vérifie s'il peut être le min
    jbe minimum

    ; Sinon c'est le nouveau max
    mov max, al
    jmp next

minimum:
    cmp al, min
    jae next
    mov min, al

next:
    inc si
    loop boucle

hlt
code ends
end start
```

Recherche sur tableau 16 bits (Mots)

```
data segment
    table dw 15, 20, 10, 11, 2, 35, 60, 30
    max dw ?
    min dw ?
    taille dw 8
data ends

code segment
    assume ds:data, cs:code

start:
    mov ax, data
    mov ds, ax

    ; On met l'adresse de la variable table dans SI
    lea si, table

    ; On met la donnée (16 bits) dont l'offset est contenu dans SI
    mov ax, [si]

    mov max, ax
    mov min, ax
    mov cx, taille
    dec cx

    ; Pour utiliser 16 bits, on doit incrémenter SI 2 fois
    ; car chaque nombre prendra 2 lignes d'adresse de 8 bits
    inc si
    inc si

boucle:
    mov ax, [si]
    cmp ax, max

    ; Si le nombre actuel est ⩽ au max, vérifie s'il peut être le min
    jbe minimum

    ; ... (Suite de la logique identique à 8 bits mais avec registres 16 bits)
    mov max, ax
    jmp next

minimum:
    cmp ax, min
    jae next
    mov min, ax

next:
    inc si
    inc si
    loop boucle

hlt
code ends
end start
```

1.4 Instructions de décalage et rotation

Exécuter les codes que vous avez développés. Conclure.

SHL et **SHR** déplacent les bits vers la gauche ou la droite et certains bits peuvent se perdre.

Alors que **ROL** et **ROR** font tourner les bits en cercle sans en perdre aucun (rotation circulaire).

```
org 100h

; Décalage logique à gauche (Shift Left)
mov al, 20
shl al, 2

; Décalage logique à droite (Shift Right)
mov al, 20
shr al, 2

; Rotation à gauche (Rotate Left)
mov al, 11110000b
rol al, 4

; Rotation à droite (Rotate Right)
mov al, 11110000b
ror al, 4

ret
```